# AWX 入門

aoki-taquan

2026 年 04 月 11 日

# 目次

1 はじめに . . . . . 6
1.1 AWX とは . . . . . 6
1.2 AWX の主な特徴 . . . . . 6
1.3 Ansible Tower との関係 . . . . . 6
1.4 対象読者 . . . . . 6
1.5 前提知識 . . . . . 7
2 インストール . . . . . 8
2.1 システム要件 . . . . . 8
2.2 インストール方法の選択 . . . . . 8
2.3 AWX Operator（Kubernetes）でのインストール . . . . . 8
2.3.1 前提条件 . . . . . 8
2.3.2 Minikube での準備 . . . . . 8
2.3.3 AWX Operator のデプロイ . . . . . 9
2.3.4 AWX インスタンスの作成 . . . . . 9
2.3.5 管理者パスワードの取得 . . . . . 10
2.4 Docker Compose でのインストール . . . . . 10
2.4.1 前提条件 . . . . . 10
2.4.2 手順 . . . . . 10
2.4.3 初期ユーザーの作成 . . . . . 10
2.5 インストール後の確認 . . . . . 10
3 基本概念 . . . . . 12
3.1 オブジェクトの全体像 . . . . . 12
3.2 組織（Organization） . . . . . 12
3.3 ユーザーとチーム . . . . . 12
3.3.1 ユーザータイプ . . . . . 12
3.3.2 チーム . . . . . 13
3.4 認証情報（Credential） . . . . . 13
3.4.1 主な認証情報タイプ . . . . . 13
3.5 プロジェクト（Project） . . . . . 13
3.6 インベントリ（Inventory） . . . . . 13
3.7 ジョブテンプレート（Job Template） . . . . . 14
3.8 ジョブ（Job） . . . . . 14
4 初期設定 . . . . . 15
4.1 Web UI へのアクセス . . . . . 15
4.2 組織の設定 . . . . . 15
4.2.1 デフォルト組織の確認 . . . . . 15
4.2.2 新しい組織の作成 . . . . . 15
4.3 ユーザーの作成 . . . . . 15
4.3.1 ローカルユーザーの追加 . . . . . 15

4.4 チームの作成 . . . . . 16
4.5 認証情報の登録 . . . . . 16
4.5.1 マシン認証情報（SSH） . . . . . 16
4.5.2 ソースコントロール認証情報 . . . . . 16
4.6 基本設定の確認 . . . . . 16
4.6.1 ジョブ設定 . . . . . 17
4.6.2 認証設定 . . . . . 17
4.6.3 ログ設定 . . . . . 17
5 プロジェクトとインベントリ . . . . . 18
5.1 プロジェクトの作成 . . . . . 18
5.1.1 Git リポジトリとの連携 . . . . . 18
5.1.2 SCM 更新オプション . . . . . 18
5.1.3 プロジェクトの同期 . . . . . 18
5.2 インベントリの作成 . . . . . 19
5.2.1 静的インベントリ . . . . . 19
5.2.2 動的インベントリ（インベントリソース） . . . . . 19
5.2.3 SCM インベントリ . . . . . 20
5.3 スマートインベントリ . . . . . 20
6 テンプレートとジョブ . . . . . 21
6.1 ジョブテンプレートの作成 . . . . . 21
6.1.1 ジョブタイプ . . . . . 21
6.2 ジョブの実行 . . . . . 21
6.2.1 手動実行 . . . . . 21
6.2.2 実行時のプロンプト . . . . . 21
6.3 ジョブの結果確認 . . . . . 22
6.3.1 ステータス . . . . . 22
6.3.2 出力タブ . . . . . 22
6.3.3 ホストステータス . . . . . 22
6.4 サーベイ機能 . . . . . 22
6.4.1 サーベイの設定 . . . . . 22
6.4.2 活用例 . . . . . 23
6.5 スケジュール実行 . . . . . 23
7 ワークフロー . . . . . 24
7.1 ワークフローの基本 . . . . . 24
7.1.1 通常のジョブテンプレートとの違い . . . . . 24
7.2 ワークフローの作成 . . . . . 24
7.3 ビジュアライザーの操作 . . . . . 24
7.3.1 ノードの追加 . . . . . 24
7.3.2 条件分岐の設定 . . . . . 24
7.3.3 収束ノード . . . . . 25
7.4 ワークフローの実行例 . . . . . 25
7.4.1 基本的なデプロイフロー . . . . . 25

7.5 承認ノード . . . . . 25
7.5.1 承認ノードの設定 . . . . . 25
7.5.2 承認の実行 . . . . . 25
7.6 ワークフローの変数 . . . . . 26
7.6.1 ワークフロー追加変数 . . . . . 26
7.6.2 ノード間の変数受け渡し . . . . . 26
8 認証とアクセス制御 . . . . . 27
8.1 RBAC（ロールベースアクセス制御） . . . . . 27
8.1.1 ロールの種類 . . . . . 27
8.1.2 権限の付与 . . . . . 27
8.1.3 権限設計のベストプラクティス . . . . . 27
8.2 認証情報タイプ . . . . . 27
8.2.1 カスタム認証情報タイプ . . . . . 27
8.3 外部認証の設定 . . . . . 28
8.3.1 LDAP 認証 . . . . . 28
8.3.2 SAML 認証 . . . . . 29
8.3.3 GitHub / Google 認証 . . . . . 29
8.4 実行環境（Execution Environment） . . . . . 29
8.4.1 デフォルト実行環境 . . . . . 29
8.4.2 カスタム実行環境の作成 . . . . . 29
9 API 活用 . . . . . 31
9.1 API の基本 . . . . . 31
9.1.1 API エンドポイント . . . . . 31
9.1.2 主要エンドポイント . . . . . 31
9.2 認証方法 . . . . . 31
9.2.1 パーソナルアクセストークン . . . . . 31
9.2.2 Basic 認証 . . . . . 32
9.3 API の使用例 . . . . . 32
9.3.1 ジョブテンプレートの一覧取得 . . . . . 32
9.3.2 ジョブの実行 . . . . . 32
9.3.3 追加変数を指定して実行 . . . . . 32
9.3.4 ジョブの状態確認 . . . . . 32
9.3.5 ジョブの完了待ち . . . . . 33
9.4 awx CLI . . . . . 33
9.4.1 インストール . . . . . 33
9.4.2 設定 . . . . . 33
9.4.3 使用例 . . . . . 33
9.5 Ansible Collection による API 操作 . . . . . 34
10 運用 Tips . . . . . 35
10.1 バックアップとリストア . . . . . 35
10.1.1 AWX Operator 環境 . . . . . 35
10.1.2 リストア . . . . . 35

10.2 アップグレード . . . . . 35
10.2.1 AWX Operator のアップグレード . . . . . 35
10.3 パフォーマンスチューニング . . . . . 36
10.3.1 ジョブの同時実行数 . . . . . 36
10.3.2 データベースの最適化 . . . . . 36
10.3.3 リソース割り当て . . . . . 36
10.4 通知テンプレート . . . . . 37
10.4.1 通知タイプ . . . . . 37
10.4.2 Slack 通知の設定例 . . . . . 37
10.4.3 通知の関連付け . . . . . 37
10.5 トラブルシューティング . . . . . 38
10.5.1 よくある問題と対処 . . . . . 38
10.5.2 ログの確認 . . . . . 38
10.5.3 デバッグ用 API . . . . . 38
10.6 セキュリティのベストプラクティス . . . . . 38

# 1 はじめに

## 1.1 AWX とは

AWX は、Ansible の自動化機能を Web ベースの UI と REST API で管理・実行できるオープンソースプラットフォームである。Red Hat が提供する商用製品「Ansible Automation Platform（旧 Ansible Tower）」のアップストリームプロジェクトに位置づけられている。

AWX を使うことで、Ansible Playbook の実行をチームで共有し、権限管理やジョブスケジュール、実行履歴の可視化といったエンタープライズ向けの運用管理機能を利用できる。

## 1.2 AWX の主な特徴

- **Web UI**：ブラウザから Playbook の実行・管理が可能
- **REST API**：すべての操作を API から自動化できる
- **RBAC（ロールベースアクセス制御）**：組織・チーム単位での権限管理
- **ジョブスケジューリング**：定期的な Playbook 実行の自動化
- **実行履歴・監査ログ**：誰が・いつ・何を実行したかを記録
- **認証情報管理**：SSH 鍵やクラウド API キーを安全に保管
- **SCM 連携**：Git リポジトリから Playbook を自動取得
- **通知機能**：Slack、メール、Webhook 等でジョブ結果を通知
- **ワークフロー**：複数のジョブを条件分岐付きで連結実行

## 1.3 Ansible Tower との関係

AWX は Ansible Tower（現 Ansible Automation Platform Controller）のオープンソース版である。主な違いは以下の通り。

| 項目 | AWX | Ansible Automation Platform |
|---|---|---|
| ライセンス | Apache 2.0 | 商用サブスクリプション |
| サポート | コミュニティ | Red Hat 公式サポート |
| リリース | 頻繁（開発版） | 安定版リリース |
| 用途 | 開発・検証・小規模運用 | 本番環境・エンタープライズ |
| 認定コンテンツ | なし | あり |

## 1.4 対象読者

- Ansible の基本的な使い方を理解している方
- チームでの Ansible 運用を効率化したい方
- 自動化の実行管理・権限管理を導入したい方
- AWX をこれから導入・検証したい方

## 1.5 前提知識

本ガイドでは以下の知識があることを前提とする。

- Ansible の基本概念（Playbook、インベントリ、モジュール）
- Linux の基本操作（コマンドライン操作）
- Docker の基礎知識（コンテナの起動・停止）
- Kubernetes の基礎知識（AWX Operator を使う場合）

# 2 インストール

## 2.1 システム要件

AWX を動作させるための推奨スペックは以下の通りである。

| 項目 | 推奨値 |
|---|---|
| CPU | 4 コア以上 |
| メモリ | 8GB 以上 |
| ディスク | 40GB 以上 |
| OS | Ubuntu 22.04 / RHEL 9 / Rocky Linux 9 等 |
| Docker | 24.0 以上（Docker Compose 利用時） |
| Kubernetes | 1.27 以上（AWX Operator 利用時） |

## 2.2 インストール方法の選択

AWX のインストール方法は主に 2 つある。

| 方法 | 特徴 | 推奨ケース |
|---|---|---|
| AWX Operator（Kubernetes） | 公式推奨。本番向き | 本番環境・チーム利用 |
| Docker Compose | 手軽に構築可能 | 開発・検証・個人利用 |

現在、AWX プロジェクトは **AWX Operator（Kubernetes）を公式推奨** としている。Docker Compose 版は開発・検証向けに提供されている。

## 2.3 AWX Operator（Kubernetes）でのインストール

### 2.3.1 前提条件

- Kubernetes クラスタ（minikube、k3s、EKS 等）が稼働していること
- `kubectl` がインストール・設定済みであること

### 2.3.2 Minikube での準備

検証環境として Minikube を使う場合の手順を示す。

```
# Minikubeのインストール（Ubuntu）
curl -LO https://storage.googleapis.com/minikube/releases/latest/minikube-linux-amd64
sudo install minikube-linux-amd64 /usr/local/bin/minikube

# クラスタの起動
minikube start --cpus=4 --memory=8g --addons=ingress
```

### 2.3.3 AWX Operator のデプロイ

```
# kustomize用のディレクトリを作成
mkdir -p awx-operator && cd awx-operator

# kustomization.yamlを作成
cat <<EOF > kustomization.yaml
apiVersion: kustomize.config.k8s.io/v1beta1
kind: Kustomization
resources:
  - github.com/ansible/awx-operator/config/default?ref=2.19.1
images:
  - name: quay.io/ansible/awx-operator
    newTag: 2.19.1
namespace: awx
EOF

# namespaceの作成とOperatorのデプロイ
kubectl create namespace awx
kubectl apply -k .
```

### 2.3.4 AWX インスタンスの作成

```
# awx-instance.yaml
apiVersion: awx.ansible.com/v1beta1
kind: AWX
metadata:
  name: awx-demo
  namespace: awx
spec:
  service_type: NodePort
  nodeport_port: 30080
```

```
# kustomization.yamlにAWXインスタンスを追加
cat <<EOF >> kustomization.yaml
  - awx-instance.yaml
EOF

# デプロイ
kubectl apply -k .

# Podの状態を確認（全てRunningになるまで待機）
kubectl get pods -n awx -w
```

### 2.3.5 管理者パスワードの取得

```
# 管理者パスワードの取得
kubectl get secret awx-demo-admin-password -n awx \
  -o jsonpath='{.data.password}' | base64 --decode; echo
```

ブラウザで `http://<ノードIP>:30080` にアクセスし、ユーザー名 `admin` と取得したパスワードでログインする。

## 2.4 Docker Compose でのインストール

### 2.4.1 前提条件

- Docker と Docker Compose がインストール済みであること
- Git がインストール済みであること

### 2.4.2 手順

```
# AWXリポジトリのクローン
git clone -b x.y.z https://github.com/ansible/awx.git
cd awx

# Docker Compose用の設定
cd tools/docker-compose

# 環境変数の設定
cp .env.example .env

# コンテナの起動
docker compose up -d

# ログの確認
docker compose logs -f
```

### 2.4.3 初期ユーザーの作成

```
# 管理者ユーザーの作成
docker compose exec awx-web awx-manage createsuperuser \
  --username admin --email admin@example.com
```

ブラウザで `http://localhost:8043` にアクセスしてログインする。

## 2.5 インストール後の確認

インストールが完了したら、以下を確認する。

1. Web UI にログインできること

2. ダッシュボードが正常に表示されること
3. **設定** > **システム** でライセンス情報やバージョンが確認できること
4. **設定** > **ジョブ** でジョブ関連の設定が表示されること

# 3 基本概念

AWX を効果的に使うには、まず主要なオブジェクトとその関係を理解する必要がある。

## 3.1 オブジェクトの全体像

AWX の主要オブジェクトは以下の通りである。

| オブジェクト | 説明 |
|---|---|
| 組織（Organization） | ユーザー・チーム・リソースをまとめる最上位の単位 |
| ユーザー（User） | AWX にログインする個人アカウント |
| チーム（Team） | 組織内のユーザーグループ。権限の付与単位 |
| 認証情報（Credential） | SSH 鍵・API キー・パスワード等の秘密情報 |
| プロジェクト（Project） | Ansible Playbook のソースコードへの SCM 参照 |
| インベントリ（Inventory） | 管理対象ホストのリスト |
| ジョブテンプレート（Job Template） | Playbook 実行の定義（プロジェクト+インベントリ+認証情報） |
| ジョブ（Job） | ジョブテンプレートの実行インスタンス |
| ワークフロー（Workflow） | 複数ジョブを連結した実行フロー |
| スケジュール（Schedule） | ジョブの定期実行設定 |
| 通知テンプレート（Notification） | ジョブ結果の通知先設定 |

## 3.2 組織（Organization）

組織は AWX における最上位の管理単位である。すべてのリソース（プロジェクト、インベントリ、認証情報等）は組織に属する。

AWX インストール直後には「Default」組織が作成されている。部署やプロジェクト単位で組織を分けることで、リソースの分離と権限管理が容易になる。

## 3.3 ユーザーとチーム

### 3.3.1 ユーザータイプ

AWX のユーザーには 3 つのタイプがある。

- **システム管理者（System Administrator）**：全権限を持つ。AWX 全体の管理が可能
- **システム監査者（System Auditor）**：全リソースの読み取りが可能。変更は不可
- **通常ユーザー（Normal User）**：明示的に付与された権限のみ利用可能

### 3.3.2 チーム

チームは組織内のユーザーグループである。チーム単位で権限を付与することで、個別ユーザーへの権限設定を省力化できる。

## 3.4 認証情報（Credential）

認証情報は、Playbook の実行や SCM へのアクセスに必要な秘密情報を安全に管理するオブジェクトである。

### 3.4.1 主な認証情報タイプ

| タイプ | 用途 |
|---|---|
| マシン（Machine） | SSH 接続や sudo 用のユーザー名・パスワード・SSH 鍵 |
| ソースコントロール（SCM） | Git リポジトリへのアクセス用認証 |
| Vault | Ansible Vault のパスワード |
| Amazon Web Services | AWS の API アクセスキー |
| Google Compute Engine | GCP のサービスアカウント |
| Microsoft Azure | Azure のサービスプリンシパル |
| VMware vCenter | vCenter へのアクセス認証 |
| Red Hat Ansible Automation Platform | Automation Hub 接続用 |

認証情報は暗号化されてデータベースに保存され、API 経由でも平文で取得することはできない。

## 3.5 プロジェクト（Project）

プロジェクトは、Ansible Playbook のソースコードを AWX に登録するオブジェクトである。通常は Git リポジトリの URL とブランチ、および SCM 認証情報を指定する。

プロジェクトの同期（SCM Update）を実行すると、最新の Playbook コードが AWX サーバーにダウンロードされる。

## 3.6 インベントリ（Inventory）

インベントリは、Ansible が管理するホスト（対象サーバー）のリストである。AWX では以下の方法でインベントリを構成できる。

- **静的インベントリ**：ホストやグループを手動で定義
- **動的インベントリ（インベントリソース）**：クラウド API 等から自動取得
- **SCM インベントリ**：Git リポジトリ内のインベントリファイルを参照

## 3.7 ジョブテンプレート（Job Template）

ジョブテンプレートは、Playbook を実行するための設定をまとめたオブジェクトである。以下の要素を組み合わせて定義する。

- **プロジェクト**：実行する Playbook の場所
- **Playbook**：プロジェクト内の具体的な Playbook ファイル
- **インベントリ**：実行対象のホスト群
- **認証情報**：実行に必要な認証情報
- **追加変数**：`extra_vars` として渡す変数

ジョブテンプレートを「起動」すると「ジョブ」が作成され、Playbook が実行される。

## 3.8 ジョブ（Job）

ジョブは、ジョブテンプレートの実行インスタンスである。各ジョブには以下の情報が記録される。

- 実行ステータス（成功・失敗・キャンセル等）
- 実行者
- 開始・終了時刻
- 標準出力（Playbook の実行ログ）
- 変更されたホストの一覧

# 4 初期設定

AWX のインストールが完了したら、まず基本的な初期設定を行う。

## 4.1 Web UI へのアクセス

ブラウザで AWX の URL にアクセスし、管理者アカウントでログインする。ログイン後、ダッシュボードが表示される。

ダッシュボードでは以下の情報を確認できる。

- 最近のジョブ実行結果
- ジョブのステータス概要（成功・失敗の割合）
- インベントリ内のホスト数

## 4.2 組織の設定

### 4.2.1 デフォルト組織の確認

左メニューの **アクセス** > **組織** から組織一覧を確認する。初期状態では「Default」組織が作成されている。

### 4.2.2 新しい組織の作成

チームや部署ごとにリソースを分離したい場合は、新しい組織を作成する。

1. 左メニューから **アクセス** > **組織** を選択
2. **追加** ボタンをクリック
3. 以下の項目を入力
   - **名前**：組織名（例：`開発チーム`）
   - **説明**：組織の説明（任意）
4. **保存** をクリック

## 4.3 ユーザーの作成

### 4.3.1 ローカルユーザーの追加

1. 左メニューから **アクセス** > **ユーザー** を選択
2. **追加** ボタンをクリック
3. 以下の項目を入力
   - **ユーザー名**：ログイン ID
   - **メールアドレス**：メール
   - **パスワード**：ログインパスワード
   - **ユーザータイプ**：通常ユーザー / システム管理者 / システム監査者
   - **組織**：所属組織

4. **保存** をクリック

## 4.4 チームの作成

1. 左メニューから **アクセス** > **チーム** を選択
2. **追加** ボタンをクリック
3. 以下の項目を入力
   - **名前**：チーム名
   - **組織**：所属組織
4. **保存** をクリック
5. **ユーザー** タブからメンバーを追加

## 4.5 認証情報の登録

AWX で最もよく使う認証情報を登録する。

### 4.5.1 マシン認証情報（SSH）

管理対象サーバーへの SSH 接続に使用する。

1. 左メニューから **リソース** > **認証情報** を選択
2. **追加** ボタンをクリック
3. 以下の項目を入力
   - **名前**：`本番サーバーSSH鍵`
   - **認証情報タイプ**：マシン
   - **ユーザー名**：SSH ユーザー名
   - **SSH 秘密鍵**：秘密鍵の内容を貼り付け
   - **秘密鍵のパスフレーズ**：必要に応じて入力
   - **権限昇格方法**：sudo
   - **権限昇格ユーザー名**：root
4. **保存** をクリック

### 4.5.2 ソースコントロール認証情報

Git リポジトリのアクセスに使用する。

1. **認証情報タイプ**：ソースコントロール
2. **ユーザー名**：Git ユーザー名またはトークン名
3. **パスワード**：パスワードまたは Personal Access Token

SSH 鍵でアクセスする場合は、SSH 秘密鍵を設定する。

## 4.6 基本設定の確認

左メニューの **設定** から、以下の項目を確認・調整する。

### 4.6.1 ジョブ設定

**設定** > **ジョブ** で以下を確認する。

- **ジョブ実行パスの分離**：ジョブ実行環境の分離設定
- **デフォルトのジョブタイムアウト**：ジョブの最大実行時間（0 で無制限）
- **ジョブ出力のサイズ制限**：大量出力時の制限値

### 4.6.2 認証設定

**設定** > **認証** では、外部認証プロバイダ（LDAP、SAML、GitHub 等）の設定を行える。初期段階ではローカル認証のみで運用し、必要に応じて外部認証を追加するのがよい。

### 4.6.3 ログ設定

**設定** > **ログ** では、外部ログ集約サービス（Splunk、Elastic Stack 等）へのログ転送を設定できる。

# 5 プロジェクトとインベントリ

ジョブテンプレートを作成するには、まずプロジェクト（Playbook の場所）とインベントリ（実行対象のホスト）を登録する必要がある。

## 5.1 プロジェクトの作成

### 5.1.1 Git リポジトリとの連携

AWX のプロジェクトは、通常 Git リポジトリを参照する。Playbook 群を管理する Git リポジトリを用意し、AWX に登録する。

1. 左メニューから **リソース** > **プロジェクト** を選択
2. **追加** ボタンをクリック
3. 以下の項目を入力

| 項目 | 設定値 |
|---|---|
| 名前 | プロジェクト名（例：`Webサーバー構築`） |
| 組織 | 所属する組織を選択 |
| ソースコントロールタイプ | Git |
| ソースコントロール URL | リポジトリの URL（例：`https://github.com/org/playbooks.git`） |
| ソースコントロールブランチ | ブランチ名（例：`main`） |
| ソースコントロール認証情報 | Git アクセス用の認証情報を選択 |

### 5.1.2 SCM 更新オプション

プロジェクト作成時に以下のオプションを設定できる。

- **起動時の更新**：ジョブ実行前に自動で SCM 同期を行う。常に最新の Playbook で実行したい場合に有効
- **ブランチの上書きを許可**：ジョブテンプレート側でブランチを指定可能にする
- **クリーンアップ**：同期時にローカルの変更を破棄する

### 5.1.3 プロジェクトの同期

プロジェクトを保存すると自動で初回同期が実行される。同期状態は以下のアイコンで確認できる。

- 緑（成功）：正常に同期完了
- 赤（失敗）：同期エラー。認証情報や URL を確認
- 灰色（未実行）：まだ同期されていない

## 5.2 インベントリの作成

### 5.2.1 静的インベントリ

手動でホストやグループを定義する方式である。

1. 左メニューから **リソース** > **インベントリ** を選択
2. **追加** > **インベントリの追加** をクリック
3. 以下の項目を入力
   - **名前**：インベントリ名（例：`本番環境`）
   - **組織**：所属する組織
4. **保存** をクリック

#### 5.2.1.1 ホストの追加

1. 作成したインベントリの **ホスト** タブを開く
2. **追加** をクリック
3. 以下を入力
   - **名前**：ホスト名または IP アドレス
   - **変数**：ホスト固有の変数（YAML 形式）

```
# ホスト変数の例
ansible_host: 192.168.1.10
ansible_port: 22
ansible_user: deploy
```

#### 5.2.1.2 グループの追加

ホストをグループ化して管理できる。

1. インベントリの **グループ** タブを開く
2. **追加** をクリック
3. グループ名を入力（例：`webservers`）
4. **ホスト** タブからグループにホストを追加

```
# グループ変数の例
http_port: 80
max_connections: 1000
```

### 5.2.2 動的インベントリ（インベントリソース）

クラウド環境のホスト情報を自動取得する場合は、インベントリソースを使う。

1. インベントリの **ソース** タブを開く
2. **追加** をクリック
3. 以下を設定

| 項目 | 設定値 |
|---|---|
| 名前 | ソース名（例：`AWS EC2`） |

| 項目 | 設定値 |
|---|---|
| ソース | Amazon EC2 / Google Compute Engine / Azure 等 |
| 認証情報 | クラウドプロバイダの認証情報 |
| 上書き | 既存ホスト情報を最新で上書きする場合に有効化 |
| 起動時の更新 | ジョブ実行前にインベントリを自動更新 |

### 5.2.3 SCM インベントリ

Git リポジトリ内のインベントリファイルを参照する方式である。Playbook と同じリポジトリでインベントリを管理できる。

1. インベントリソースの **ソース** に「プロジェクトからソース」を選択
2. **プロジェクト** でインベントリファイルを含むプロジェクトを選択
3. **インベントリファイル** でファイルパスを指定

## 5.3 スマートインベントリ

スマートインベントリは、ホストのファクト（属性）に基づいて動的にホストをフィルタリングする機能である。

1. **追加** > **スマートインベントリの追加** を選択
2. **ホストフィルター** に検索条件を入力

```
# フィルター例：OSがCentOSのホスト
ansible_facts.ansible_distribution:CentOS
```

複数のインベントリにまたがるホストを横断的に管理したい場合に有用である。

# 6 テンプレートとジョブ

ジョブテンプレートは、AWX で Playbook を実行するための中核となるオブジェクトである。

## 6.1 ジョブテンプレートの作成

1. 左メニューから **リソース** > **テンプレート** を選択
2. **追加** > **ジョブテンプレートの追加** をクリック
3. 以下の項目を設定

| 項目 | 説明 |
|---|---|
| 名前 | テンプレート名（例：`Webサーバーセットアップ`） |
| ジョブタイプ | 実行（Run）/ チェック（Check） |
| インベントリ | 対象ホストのインベントリ |
| プロジェクト | Playbook が含まれるプロジェクト |
| Playbook | 実行する Playbook ファイル |
| 認証情報 | 実行に使用する認証情報 |
| 実行環境 | Playbook を実行するコンテナ環境 |
| フォーク数 | 並列実行数（デフォルト：0 で Ansible 設定に従う） |
| 制限 | 実行対象のホストパターン（任意） |
| 詳細度 | 出力の詳細レベル（0〜5） |
| 追加変数 | `extra_vars` として渡す変数（YAML 形式） |

### 6.1.1 ジョブタイプ

- **実行（Run）**：Playbook を通常実行する
- **チェック（Check）**：ドライランモード。実際の変更は行わず、変更が発生するかを確認する

## 6.2 ジョブの実行

### 6.2.1 手動実行

ジョブテンプレートの一覧画面、または詳細画面からロケットアイコン（起動ボタン）をクリックすると、ジョブが実行される。

### 6.2.2 実行時のプロンプト

ジョブテンプレートの設定で「起動時にプロンプト」を有効にした項目は、実行時に値を指定できる。以下の項目でプロンプトが可能である。

- インベントリ
- 認証情報
- 制限（Limit）
- SCM ブランチ

- 追加変数
- ジョブタイプ
- 詳細度
- タグ / スキップタグ

## 6.3 ジョブの結果確認

ジョブの実行結果画面では、以下の情報を確認できる。

### 6.3.1 ステータス

| ステータス | 意味 |
|---|---|
| Successful | すべてのタスクが正常に完了 |
| Failed | 1 つ以上のタスクが失敗 |
| Error | Playbook の実行自体がエラー |
| Canceled | ユーザーによりキャンセルされた |
| Pending | 実行待ち |
| Running | 実行中 |

### 6.3.2 出力タブ

Playbook の標準出力がリアルタイムで表示される。タスクごとに折りたたみ可能で、各ホストの実行結果（ok / changed / failed / skipped）を確認できる。

### 6.3.3 ホストステータス

ジョブの **ホストステータス** では、各ホストの結果サマリーが表示される。

- **OK**：変更なしで正常完了したタスク数
- **Changed**：変更が発生したタスク数
- **Failed**：失敗したタスク数
- **Skipped**：スキップされたタスク数

## 6.4 サーベイ機能

サーベイは、ジョブ実行時にユーザーからパラメータを入力させるフォーム機能である。`extra_vars` に値を渡す仕組みで、Playbook をパラメータ化して再利用性を高められる。

### 6.4.1 サーベイの設定

1. ジョブテンプレートの **サーベイ** タブを開く
2. **サーベイの追加** をクリック
3. 以下の項目を設定

| 項目 | 説明 |
|---|---|
| 質問 | ユーザーに表示する質問文 |

| 項目 | 説明 |
|---|---|
| 回答の変数名 | Playbook で参照する変数名 |
| 回答タイプ | テキスト / 選択 / 数値 / パスワード等 |
| デフォルト値 | デフォルトの回答値 |
| 必須 | 回答を必須にするか |

### 6.4.2 活用例

```
# Playbookでサーベイ変数を使用
- name: Deploy application
  hosts: "{{ target_env }}"
  vars:
    app_version: "{{ deploy_version }}"
  tasks:
    - name: Deploy version {{ deploy_version }}
      ansible.builtin.debug:
        msg: "Deploying to {{ target_env }}"
```

サーベイで `target_env`（環境選択）と `deploy_version`（バージョン）を入力させることで、同じテンプレートで異なる環境・バージョンへのデプロイが可能になる。

## 6.5 スケジュール実行

ジョブテンプレートにスケジュールを設定すると、指定した日時に自動で実行される。

1. ジョブテンプレートの **スケジュール** タブを開く
2. **追加** をクリック
3. 以下を設定
   - **名前**：スケジュール名
   - **開始日時**：最初の実行日時
   - **繰り返し**：繰り返しパターン（毎日・毎週・毎月等）
   - **タイムゾーン**：実行タイムゾーン

定期的なパッチ適用やコンプライアンスチェックなどに活用できる。

# 7 ワークフロー

ワークフロージョブテンプレートは、複数のジョブテンプレートを連結して実行する機能である。条件分岐や並列実行を組み合わせた複雑な自動化フローを構築できる。

## 7.1 ワークフローの基本

### 7.1.1 通常のジョブテンプレートとの違い

| 項目 | ジョブテンプレート | ワークフロー |
|---|---|---|
| 実行内容 | 単一の Playbook | 複数のジョブを連結 |
| 条件分岐 | 不可 | 成功/失敗/常に で分岐可能 |
| 並列実行 | 不可 | 複数ジョブの同時実行可能 |
| 承認ステップ | なし | 承認ノードで一時停止可能 |
| 異なるインベントリ | 1 つ | ノードごとに変更可能 |

## 7.2 ワークフローの作成

1. **リソース** > **テンプレート** から **追加** > **ワークフロージョブテンプレートの追加** を選択
2. 名前と組織を入力して **保存**
3. **ビジュアライザー** タブでフローを構築

## 7.3 ビジュアライザーの操作

ワークフロービジュアライザーは、フローをグラフィカルに構築するエディタである。

### 7.3.1 ノードの追加

1. **開始** ノードの ＋ ボタンをクリック
2. ノードタイプを選択
   - **ジョブテンプレート**：Playbook の実行
   - **ワークフロージョブテンプレート**：別のワークフローの呼び出し
   - **プロジェクト同期**：プロジェクトの SCM 更新
   - **インベントリ同期**：インベントリソースの更新
   - **承認**：人手による承認待ち
3. 実行するテンプレート等を選択して **保存**

### 7.3.2 条件分岐の設定

ノード間の接続には 3 種類の条件を設定できる。

- **成功時（On Success）**：前のノードが成功した場合に実行（緑の線）
- **失敗時（On Failure）**：前のノードが失敗した場合に実行（赤の線）

- **常に（Always）**：前のノードの結果に関わらず実行（青の線）

### 7.3.3 収束ノード

複数のノードから 1 つのノードに収束させることも可能である。収束ノードは、接続元のすべてのノードの実行が完了してから実行される。条件の組み合わせにより、AND 条件や OR 条件の表現が可能である。

## 7.4 ワークフローの実行例

### 7.4.1 基本的なデプロイフロー

```
開始
 └─[成功]──→ コードデプロイ
              ├─[成功]──→ ヘルスチェック
              │            ├─[成功]──→ 完了通知
              │            └─[失敗]──→ ロールバック
              │                         └─[常に]──→ 障害通知
              └─[失敗]──→ デプロイ失敗通知
```

このフローでは以下の動作をする。

1. コードデプロイを実行
2. デプロイ成功時：ヘルスチェックを実行
3. ヘルスチェック成功時：完了通知を送信
4. ヘルスチェック失敗時：ロールバックを実行し、障害通知を送信
5. デプロイ失敗時：デプロイ失敗通知を送信

## 7.5 承認ノード

承認ノードは、ワークフローの途中で人手による承認を待つ特殊なノードである。本番環境へのデプロイ前の確認などに利用する。

### 7.5.1 承認ノードの設定

1. ビジュアライザーでノード追加時に **承認** タイプを選択
2. 以下を設定
   - **名前**：承認ステップの名前（例：`本番デプロイ承認`）
   - **説明**：承認者への説明メッセージ
   - **タイムアウト**：承認待ちのタイムアウト時間（0 で無制限）

### 7.5.2 承認の実行

ワークフローが承認ノードに到達すると、以下の動作をする。

1. ワークフローが一時停止
2. 承認権限を持つユーザーに通知（通知テンプレート設定時）

3. 承認者が Web UI から **承認** または **拒否** を選択
4. 承認：後続の成功ノードが実行される
5. 拒否：後続の失敗ノードが実行される
6. タイムアウト：失敗として扱われる

## 7.6 ワークフローの変数

### 7.6.1 ワークフロー追加変数

ワークフロー全体で使用する変数を設定できる。各ノードのジョブテンプレートから参照可能である。

### 7.6.2 ノード間の変数受け渡し

Ansible の `set_stats` モジュールを使うことで、ノード間で変数を受け渡せる。

```
# 前段のPlaybookで変数をセット
- name: Set artifact for next node
  ansible.builtin.set_stats:
    data:
      deployed_version: "1.2.3"
      deploy_timestamp: "{{ ansible_date_time.iso8601 }}"
```

後続のノードでは、これらの変数を `extra_vars` として参照できる。

# 8 認証とアクセス制御

AWX は、エンタープライズ環境向けの細かなアクセス制御機能を備えている。

## 8.1 RBAC（ロールベースアクセス制御）

AWX の RBAC は、オブジェクトごとにロール（権限）を割り当てる仕組みである。

### 8.1.1 ロールの種類

各オブジェクトには以下のようなロールが定義されている。

| ロール | 権限 |
|---|---|
| Admin（管理者） | オブジェクトの読み取り・編集・削除・権限付与が可能 |
| Use（使用） | ジョブテンプレート等でオブジェクトを参照可能 |
| Update（更新） | プロジェクトやインベントリの同期を実行可能 |
| Execute（実行） | ジョブテンプレートの実行が可能 |
| Read（読み取り） | オブジェクトの閲覧が可能 |
| Approve（承認） | ワークフローの承認ノードを承認可能 |

### 8.1.2 権限の付与

権限はユーザーまたはチームに対して付与できる。

1. 対象オブジェクト（プロジェクト、インベントリ等）の詳細画面を開く
2. **アクセス** タブを選択
3. **追加** をクリック
4. ユーザーまたはチームを選択
5. ロールを選択して **保存**

### 8.1.3 権限設計のベストプラクティス

- **チーム単位で権限を付与**：個別ユーザーではなくチームに権限を設定し、メンバーの追加・削除で権限管理を行う
- **最小権限の原則**：必要最低限の権限のみ付与する
- **組織でリソースを分離**：部署やプロジェクト単位で組織を分け、リソースの可視性を制御する

## 8.2 認証情報タイプ

### 8.2.1 カスタム認証情報タイプ

AWX には標準の認証情報タイプが用意されているが、独自のタイプを作成することも可能である。

1. **管理** > **認証情報タイプ** を選択
2. **追加** をクリック
3. 以下を定義

**入力設定**（JSON 形式）：認証情報の入力フィールドを定義

```
{
  "fields": [
    {
      "id": "api_token",
      "type": "string",
      "label": "API Token",
      "secret": true
    },
    {
      "id": "api_url",
      "type": "string",
      "label": "API URL"
    }
  ],
  "required": ["api_token", "api_url"]
}
```

**インジェクター設定**（JSON 形式）：Playbook に値を渡す方法を定義

```
{
  "extra_vars": {
    "custom_api_token": "{{ api_token }}",
    "custom_api_url": "{{ api_url }}"
  }
}
```

## 8.3 外部認証の設定

### 8.3.1 LDAP 認証

Active Directory や OpenLDAP と連携する設定である。

1. **設定** > **認証** > **LDAP** を選択
2. 以下の主要項目を設定

| 項目 | 設定値の例 |
|---|---|
| LDAP サーバー URI | `ldaps://ldap.example.com:636` |
| バインド DN | `cn=awx-bind,ou=service,dc=example,dc=com` |
| バインドパスワード | バインドユーザーのパスワード |
| ユーザー検索 | `ou=users,dc=example,dc=com` / `SCOPE_SUBTREE` / `(sAMAccountName=%(user)s)` |
| グループ検索 | `ou=groups,dc=example,dc=com` / `SCOPE_SUBTREE` / `(objectClass=group)` |
| ユーザーフラグ(管理者) | 管理者にマッピングする LDAP グループ |

### 8.3.2 SAML 認証

SAML 2.0 に対応した IdP（Okta、Azure AD 等）と連携できる。

1. **設定** > **認証** > **SAML** を選択
2. 以下の主要項目を設定
   - **エンティティ ID**：AWX の SAML エンティティ ID
   - **アサーションコンシューマーサービス URL**：AWX の ACS URL
   - **IdP メタデータ URL**：IdP のメタデータ URL
   - **組織マッピング**：SAML 属性と AWX 組織のマッピング
   - **チームマッピング**：SAML 属性と AWX チームのマッピング

### 8.3.3 GitHub / Google 認証

OAuth2 ベースのソーシャル認証も設定可能である。

1. **設定** > **認証** > **GitHub**（または Google）を選択
2. OAuth アプリケーションの Client ID と Client Secret を設定
3. 組織マッピングを設定

## 8.4 実行環境（Execution Environment）

実行環境は、Playbook 実行に必要な Ansible バージョン・コレクション・Python ライブラリをパッケージ化したコンテナイメージである。

### 8.4.1 デフォルト実行環境

AWX にはデフォルトの実行環境が含まれている。カスタムコレクションや Python パッケージが必要な場合は、独自の実行環境を作成する。

### 8.4.2 カスタム実行環境の作成

`ansible-builder` ツールを使用して作成する。

```
# ansible-builderのインストール
pip install ansible-builder

# 実行環境の定義ファイル
cat <<EOF > execution-environment.yml
version: 3
dependencies:
  galaxy:
    collections:
      - amazon.aws
      - community.general
  python:
    - boto3
    - botocore
  system:
```

```
    - gcc
EOF

# ビルド
ansible-builder build --tag my-custom-ee:latest
```

作成したイメージをコンテナレジストリにプッシュし、AWX の **管理** > **実行環境** から登録する。

# 9 API 活用

AWX のすべての機能は REST API を通じて利用可能である。Web UI で行える操作はすべて API でも実行できる。

## 9.1 API の基本

### 9.1.1 API エンドポイント

AWX の API は以下のベース URL で提供される。

```
https://<AWXホスト>/api/v2/
```

ブラウザでこの URL にアクセスすると、利用可能なエンドポイントの一覧が表示される（API ブラウザ）。

### 9.1.2 主要エンドポイント

| エンドポイント | 説明 |
|---|---|
| `/api/v2/organizations/` | 組織の一覧・作成 |
| `/api/v2/users/` | ユーザーの一覧・作成 |
| `/api/v2/projects/` | プロジェクトの一覧・作成 |
| `/api/v2/inventories/` | インベントリの一覧・作成 |
| `/api/v2/credentials/` | 認証情報の一覧・作成 |
| `/api/v2/job_templates/` | ジョブテンプレートの一覧・作成 |
| `/api/v2/jobs/` | ジョブの一覧・詳細 |
| `/api/v2/workflow_job_templates/` | ワークフローの一覧・作成 |

## 9.2 認証方法

### 9.2.1 パーソナルアクセストークン

API アクセスには、パーソナルアクセストークン（PAT）の使用が推奨される。

#### 9.2.1.1 トークンの作成（Web UI）

1. 右上のユーザーアイコン > **ユーザー詳細** を選択
2. **トークン** タブを開く
3. **追加** をクリック
4. スコープ（Read / Write）を選択して **保存**
5. 表示されたトークンを安全に保管（再表示不可）

#### 9.2.1.2 トークンの使用

```
# Authorizationヘッダーにトークンを指定
curl -H "Authorization: Bearer <トークン>" \
  https://awx.example.com/api/v2/me/
```

### 9.2.2 Basic 認証

テスト目的では Basic 認証も使用可能であるが、本番環境ではトークン認証を推奨する。

```
curl -u "admin:password" \
  https://awx.example.com/api/v2/me/
```

## 9.3 API の使用例

### 9.3.1 ジョブテンプレートの一覧取得

```
curl -s -H "Authorization: Bearer $TOKEN" \
  https://awx.example.com/api/v2/job_templates/ | jq '.results[] | {id, name}'
```

### 9.3.2 ジョブの実行

```
# ジョブテンプレートID 5 を実行
curl -X POST \
  -H "Authorization: Bearer $TOKEN" \
  -H "Content-Type: application/json" \
  https://awx.example.com/api/v2/job_templates/5/launch/
```

### 9.3.3 追加変数を指定して実行

```
curl -X POST \
  -H "Authorization: Bearer $TOKEN" \
  -H "Content-Type: application/json" \
  -d '{"extra_vars": {"target_env": "staging", "version": "1.2.3"}}' \
  https://awx.example.com/api/v2/job_templates/5/launch/
```

### 9.3.4 ジョブの状態確認

```
# ジョブID 42 の状態を確認
curl -s -H "Authorization: Bearer $TOKEN" \
  https://awx.example.com/api/v2/jobs/42/ | jq '{status, started, finished}'
```

### 9.3.5 ジョブの完了待ち

```
# ジョブの完了をポーリングで待機
JOB_ID=42
while true; do
  STATUS=$(curl -s -H "Authorization: Bearer $TOKEN" \
    https://awx.example.com/api/v2/jobs/$JOB_ID/ | jq -r '.status')
  echo "Status: $STATUS"
  case $STATUS in
    successful|failed|error|canceled) break ;;
  esac
  sleep 5
done
```

## 9.4 awx CLI

`awx` コマンドラインツールを使うと、シェルから AWX を操作できる。

### 9.4.1 インストール

```
pip install awxkit
```

### 9.4.2 設定

```
export TOWER_HOST=https://awx.example.com
export TOWER_USERNAME=admin
export TOWER_PASSWORD=password
# またはトークン認証
export TOWER_OAUTH_TOKEN=<トークン>
```

### 9.4.3 使用例

```
# ジョブテンプレートの一覧
awx job_templates list --all

# ジョブの実行
awx job_templates launch 5 --extra_vars '{"env": "prod"}'

# ジョブの状態確認
awx jobs get 42

# インベントリの一覧
awx inventory list

# プロジェクトの同期
awx projects update 3
```

## 9.5 Ansible Collection による API 操作

`ansible.controller` コレクション（旧 `awx.awx`）を使うと、Ansible の Playbook から AWX を操作できる。

```yaml
- name: AWXのリソースを構成
  hosts: localhost
  collections:
    - ansible.controller
  tasks:
    - name: プロジェクトを作成
      project:
        name: "My Project"
        organization: "Default"
        scm_type: git
        scm_url: "https://github.com/org/playbooks.git"
        controller_host: "https://awx.example.com"
        controller_oauthtoken: "{{ awx_token }}"

    - name: ジョブテンプレートを作成
      job_template:
        name: "Deploy App"
        project: "My Project"
        playbook: "deploy.yml"
        inventory: "Production"
        controller_host: "https://awx.example.com"
        controller_oauthtoken: "{{ awx_token }}"
```

この方法を使えば、AWX 自体の構成をコード化（Configuration as Code）できる。

# 10 運用 Tips

AWX を安定して運用するためのヒントやトラブルシューティングをまとめる。

## 10.1 バックアップとリストア

### 10.1.1 AWX Operator 環境

AWX Operator では、`AWXBackup` リソースを作成することでバックアップを取得できる。

```yaml
# awx-backup.yaml
apiVersion: awx.ansible.com/v1beta1
kind: AWXBackup
metadata:
  name: awx-backup-20260315
  namespace: awx
spec:
  deployment_name: awx-demo
  backup_pvc_namespace: awx
```

```bash
# バックアップの実行
kubectl apply -f awx-backup.yaml

# バックアップ状態の確認
kubectl get awxbackup -n awx
```

### 10.1.2 リストア

```yaml
# awx-restore.yaml
apiVersion: awx.ansible.com/v1beta1
kind: AWXRestore
metadata:
  name: awx-restore
  namespace: awx
spec:
  deployment_name: awx-demo
  backup_name: awx-backup-20260315
```

```bash
kubectl apply -f awx-restore.yaml
```

## 10.2 アップグレード

### 10.2.1 AWX Operator のアップグレード

1. `kustomization.yaml` のバージョンタグを更新

2. `kubectl apply -k .` で適用
3. Pod が再作成されるのを確認

```
# kustomization.yamlのバージョンを更新後
kubectl apply -k .

# Podの再起動を監視
kubectl get pods -n awx -w
```

アップグレード前には必ずバックアップを取得すること。

## 10.3 パフォーマンスチューニング

### 10.3.1 ジョブの同時実行数

AWX の設定で、同時に実行できるジョブの最大数を調整できる。

- **設定** > **ジョブ** > **最大同時ジョブ数**：ノードあたりの同時実行数
- フォーク数の合計がサーバーリソースを超えないよう注意

### 10.3.2 データベースの最適化

長期間運用すると、ジョブの実行履歴やイベントデータが蓄積される。定期的なクリーンアップが必要である。

```
# 古いジョブデータの削除（AWX管理コマンド）
awx-manage cleanup_jobs --days=90

# 古いアクティビティストリームの削除
awx-manage cleanup_activitystream --days=180
```

**設定** > **ジョブ** の「ジョブ実行データの保持日数」でも自動クリーンアップを設定できる。

### 10.3.3 リソース割り当て

Kubernetes 環境では、AWX の Pod に十分なリソースを割り当てる。

```
# AWXインスタンスのリソース設定例
apiVersion: awx.ansible.com/v1beta1
kind: AWX
metadata:
  name: awx-demo
  namespace: awx
spec:
  web_resource_requirements:
    requests:
      cpu: "500m"
      memory: "1Gi"
    limits:
```

```
      cpu: "2000m"
      memory: "4Gi"
  task_resource_requirements:
    requests:
      cpu: "500m"
      memory: "1Gi"
    limits:
      cpu: "2000m"
      memory: "4Gi"
```

## 10.4 通知テンプレート

ジョブの結果を外部サービスに通知する設定である。

### 10.4.1 通知タイプ

| タイプ | 説明 |
|---|---|
| Email | メール通知 |
| Slack | Slack チャンネルへの通知 |
| Webhook | 任意の URL へ HTTP リクエスト |
| PagerDuty | PagerDuty へのインシデント通知 |
| Grafana | Grafana のアノテーション |
| IRC | IRC チャンネルへの通知 |
| Mattermost | Mattermost への通知 |
| Rocket.Chat | Rocket.Chat への通知 |

### 10.4.2 Slack 通知の設定例

1. **管理** > **通知テンプレート** から **追加** をクリック
2. タイプに **Slack** を選択
3. **Webhook URL**：Slack Incoming Webhook の URL
4. **チャンネル**：通知先チャンネル（例：`#ansible-alerts`）
5. **通知をテスト** で送信テスト

### 10.4.3 通知の関連付け

通知テンプレートをジョブテンプレートやワークフローに関連付ける。

1. ジョブテンプレートの **通知** タブを開く
2. 通知テンプレートを選択
3. トリガー条件を設定（開始時 / 成功時 / 失敗時）

## 10.5 トラブルシューティング

### 10.5.1 よくある問題と対処

| 問題 | 対処 |
|---|---|
| プロジェクト同期が失敗する | SCM URL と認証情報を確認。ネットワーク疎通をテスト |
| ジョブが `Pending` のまま | 実行ノードの状態を確認。同時実行数の上限を確認 |
| 認証情報エラー | 認証情報の内容を再確認。SSH 鍵のフォーマットを確認 |
| インベントリ同期エラー | クラウド認証情報の権限を確認。IAM ポリシーを見直し |
| Web UI が遅い | データベースのクリーンアップ。リソース割り当ての見直し |
| Pod が再起動を繰り返す | リソース不足の可能性。`kubectl describe pod` でイベントを確認 |

### 10.5.2 ログの確認

```
# AWX Operator環境でのログ確認
kubectl logs -f deployment/awx-demo-web -n awx -c awx-web
kubectl logs -f deployment/awx-demo-task -n awx -c awx-task

# Receptor（実行ノード）のログ
kubectl logs -f deployment/awx-demo-task -n awx -c awx-receptor
```

### 10.5.3 デバッグ用 API

```
# システムの状態確認
curl -H "Authorization: Bearer $TOKEN" \
  https://awx.example.com/api/v2/ping/

# 実行ノードの状態
curl -H "Authorization: Bearer $TOKEN" \
  https://awx.example.com/api/v2/instances/
```

## 10.6 セキュリティのベストプラクティス

- **HTTPS の有効化**：本番環境では必ず TLS/SSL を設定する
- **デフォルトパスワードの変更**：初期管理者パスワードを速やかに変更
- **外部認証の利用**：LDAP/SAML での認証を推奨
- **定期的なトークンローテーション**：API トークンを定期的に更新
- **認証情報の最小権限化**：マシン認証情報には sudo 制限付きのユーザーを使用
- **監査ログの監視**：不審な操作がないか定期的に確認